Package 'CrossingTools'

January 28, 2026

Title Tools for Cross Optimization in Plant Breeding Programs

Version 1.0.1

Description Mate allocation is a critical component of plant breeding programs, aiming to generate superior progenies and improved populations over time. While numerous methodologies for mate allocation exist, few software tools offer a unified and scalable framework that integrates these methodologies in a user-friendly manner. 'CrossingTools' addresses this gap by providing a flexible, efficient solution for optimizing mating decisions across a wide range of breeding schemes. The package supports genomic BLUP and selection indices for various variance structures and implements multiple selection criteria to evaluate cross potential, including expected mean, optimal haploid value (OHV), and superior progeny value (SPV). Different methods are provided for calculating additive and dominance segregation variances for both inbred and heterozygous parents, making 'CrossingTools' suitable for a wide range of breeding strategies. After criterion calculation, users can rank potential crosses or apply the built-in optimal crossing selection routine, which balances genetic gain and diversity to support long-term improvement. The package combines R's scripting flexibility with high-performance C++ routines via 'Rcpp' and 'RcppArmadillo', ensuring scalability for large populations with dense marker data.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE, roclets = c(` `collate", ` `namespace", ` `rd"))

ByteCompile true

RoxygenNote 7.3.3

Depends R (>= 4.0.0)

Imports ggplot2,
ggridges,
Rcpp,
RcppParallel

LinkingTo Rcpp,
RcppArmadillo,
RcppParallel

SystemRequirements C++17

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

URL https://github.com/svenernstW/CrossingTools

BugReports https://github.com/svenernstW/CrossingTools/issues

R topics documented:

`get_marker_effects` *Backsolve marker effects (mu) from individual effects (g)*

Description

Computes marker effects $\mu = (scalingFactor * M' * G^{-1}) * g$.

Usage

```
get_marker_effects(marker.mat, G.mat, effects, scaling.factor, n.Threads = 4L)
```

Arguments

`marker.mat`	Numeric matrix (n x p): marker/genotype matrix (individuals in rows, markers in columns).
`G.mat`	Numeric matrix (n x n): relationship matrix among individuals.
`effects`	Numeric (n x k) or length-n vector: individual effects (one or more traits).
`scaling.factor`	Numeric scalar used to scale the GRM.
`n.Threads`	Integer >= 1; OpenMP threads (if enabled at compile time). Default: `4L`.

Value

A numeric p x k matrix of marker effects (`mu_matrix`).

`get_optimal_haploid_value` *Optimal Haploid Value (OHV) for proposed crosses*

Description

Computes the Optimal Haploid Value (OHV; Daetwyler et al.) for each proposed cross using block-wise local genomic estimated breeding values (GEBVs).

Usage

```
get_optimal_haploid_value(
  crosses,
  haplotype.blocks = NULL,
  marker.mat,
  effects,
  weights = NULL,
  nthreads = 4L
)
```

Arguments

`crosses`	A matrix or data.frame (n_crosses x 2 for two way crosses or n_crosses x 4 for four way crosses) specifying the parents for each proposed cross. Entries may be either: • integer indices referring to rows of `marker.mat`, or • character identifiers matching `rownames(marker.mat)`.
`haplotype.blocks`	Optional data.frame defining haplotype blocks with columns: • `block`: block identifier (integer, character, or factor). Markers sharing the same value belong to the same haplotype block. • `site`: marker identifier within `marker.mat`, given either as an integer column index (1..ncol(marker.mat)) or as a marker name matching `names(marker.mat)`. Each marker may appear in at most one block.
`marker.mat`	Numeric genotype or marker matrix with individuals in rows and markers in columns.
`effects`	Numeric matrix of marker effects with `nrow(effects) == ncol(marker.mat)` with traits in columns.
`weights`	Optional numeric vector of length `ncol(effects)`. If supplied, a weighted index is computed as a linear combination of the trait-specific OHVs.
`nthreads`	Integer >= 1. Number of threads used for parallel computation.

Details

For each haplotype block, a local GEBV is calculated for every parent as the sum of marker genotypes within the block weighted by their corresponding marker effects. The OHV of a cross is then obtained by summing, across blocks, the maximum of the two parents' local block GEBVs.

If `haplotype.blocks` is not supplied , each marker column of `marker.mat` is treated as an independent block.

Value

If `weights` is `NULL`, a data.frame containing the cross definition columns followed by `OHV.1`, `OHV.2`, `...` for each trait.

If `weights` is supplied, a list with elements:

- `cross.df`: data.frame of crosses and trait-specific OHVs
- `index.df`: data.frame of crosses and the weighted OHV index

`get_segvar_inbred` *Segregation variance for inbred lines from specified crosses*

Description

Calculate expected genomic estimated breeding values (EGEBV), segregation variance, and superior progeny value (SPV) for doubled haploid or recombinant inbred lines #' derived after `t` rounds of random mating. works for two way, three way and four way crosses

Usage

```
get_segvar_inbred(
  crosses,
  genetic.map,
  marker.mat,
  effects,
  t,
  intensity = NULL,
  type = "DH",
  covariance = FALSE,
  weights = NULL,
  method = 1,
  nthreads = 4L
)
```

Arguments

`crosses`	A matrix or data.frame (n_crosses x 2 for two way crosses or n_crosses x 4 for four way crosses) specifying the parents for each proposed cross. Entries may be either: • integer indices referring to rows of `marker.mat`, or • character identifiers matching `rownames(marker.mat)`.
`genetic.map`	A data.frame with columns: • `site`: integer marker index (1..ncol(marker.mat)) • `chr`: chromosome identifier (numeric) • `pos`: position on the chromosome in morgan All markers in `marker.mat` must appear in `genetic.map$site`.
`marker.mat`	Numeric marker matrix (markers in columns, genotypes in rows) with entries in `c(0, 2)` corresponding to `c("AA", "BB")`.

effects	Numeric matrix of marker effects (markers in rows, traits in columns). Must have nrow(effects) == ncol(marker.mat).
t	Integer. Number of random-mating generations before DH creation.
intensity	Double. Standardized selection differential, efaults to 1.
type	intended offspring typew can be "RIL" for recombinant inbred lines or "DH" for double haploid lines.
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
weights	Numeric vector of length ncol(effects) with fixed trait weights. If supplied the function calculates index values for each cross. Only works if covariance = TRUE; otherwise ignored.
method	Character. Which method to use; one of c(1, 2) for Lehermeier or Osthushenrich.
nthreads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross.df (a data.frame) whose columns are named EGBEV1, var1, SPV1, EGEBV2, ... and a list with element covariances. If covariance = TRUE and calculate.index = TRUE, additionally an element index.df (a data.frame) with IDX, VAR.IDX, SPV.IDX. If covariance = FALSE, a data.frame with the same columns EGEBV1, var1, SPV1, EGBEV2, ...

---

get_segvar_outcross *Additive and dominance segregation variance of families from specified crosses (Wolfe)*

Description

Calculate expected genomic estimated breeding value (EGBV), expected total genetic value (ETGV), additive and dominance segregation variance, superior progeny value with additive variance (SPV), and superior progeny value including dominance variance (TSPV) for F1 families.

Usage

```
get_segvar_outcross(
  crosses,
  genetic.map,
  hap.mat1,
  hap.mat2,
  effects.A,
  effects.D,
  intensity = NULL,
  covariance = FALSE,
  weights = NULL,
  nthreads = 4L
)
```

Arguments

crosses	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into hap.mat1/hap.mat2) specifying the crosses.
genetic.map	A data.frame with columns: • site: integer marker index (1..ncol(hap.mat1)) • chr: chromosome identifier (numeric or factor) • pos: position on the chromosome in morgan All markers in hap.mat1/hap.mat2 must appear in genetic.map$site.
hap.mat1	Numeric haplotype matrix (individuals x markers) for the first haplotype, entries in c(0, 1) for c("A", "B").
hap.mat2	Numeric haplotype matrix (individuals x markers) for the second haplotype, entries in c(0, 1) for c("A", "B").
effects.A	Numeric matrix of additive marker effects (markers in rows, traits in columns). Must have nrow(effects.A) == ncol(hap.mat1).
effects.D	Numeric matrix of dominance marker effects (markers in rows, traits in columns). Must have nrow(effects.D) == ncol(hap.mat1) and ncol(effects.D) == ncol(effects.A).
intensity	Double. Standardized selection differential (used for SPV), defaults to 1.
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
weights	Numeric vector of length ncol(effects.A) with fixed trait weights. If supplied the function calculates index values for each cross. Only works if covariance = TRUE; otherwise ignored.
nthreads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross.df (a data.frame) whose columns are named EGEBV1, ETGV1, var.A1, SPV1, var.D1, TSPV1, EGEBV2, ... and covariance matrices per cross. If covariance = TRUE and calculate.index = TRUE, additionally an element index.df (a data.frame) with IDX.A, var.IDX.A, SPV.IDX, IDX.AD, var.IDX.AD, SPV.IDX.AD. If covariance = FALSE, a data.frame with the same columns.

---

make_cross_plan *Creation of a plan of all potential crosses*

Description

Generates a crossing plan containing all possible pairwise crosses among a set of candidate genotypes. The function supports both unsexed (symmetric) crossing designs and sexed designs with distinct male and female candidate sets.

Usage

```
make_cross_plan(parents = NULL, self = FALSE, males = NULL, females = NULL)
```

Arguments

parents	A vector of genotype identifiers defining the candidate parents for an unsexed crossing design. Identifiers may be integers or character strings (for example, row names of a genotype matrix). Must contain at least two unique entries.
self	Logical. If TRUE and parents is supplied, self-crosses (parent × itself) are included in addition to all pairwise crosses. Ignored when males and females are supplied.
males	A vector of genotype identifiers defining male parents in a sexed crossing design. Identifiers may be integers or character strings. Ignored if parents is supplied.
females	A vector of genotype identifiers defining female parents in a sexed crossing design. Identifiers may be integers or character strings. Ignored if parents is supplied.

Details

If parents is supplied, all unique unordered pairs of parents are returned. Optionally, self-crosses can be included by setting self = TRUE.

Alternatively, if males and females are supplied, all possible male–female combinations are returned. In this case, the self argument is ignored.

Value

A two-column data.frame defining the crossing plan.

- If parents is supplied, the columns are named parent1 and parent2 and contain all unique pairwise combinations (and optionally self-crosses).
- If males and females are supplied, the columns are named male and female and contain all male–female combinations.

The output preserves the type of the supplied identifiers (integer or character), with factors automatically coerced to character.

---

make_index *Calculation of a selection index*

Description

Calculates a multi-trait selection index from genomic or phenotypic BLUPs. The function supports two alternative index formulations:

Usage

```
make_index(
  effects,
  var.mat = NULL,
  gains = NULL,
  weights = NULL,
  nthreads = 4L
)
```

Arguments

`effects`	A numeric matrix ($n_{genotype} \times n_{trait}$) of multi-trait BLUPs.
`var.mat`	Optional variance or covariance matrix used for the desired gains index. If provided, must be one of: • A numeric matrix of dimension $(n_{genotype} \times n_{trait}) \times (n_{genotype} \times n_{trait})$ containing the posterior variance $\mathrm{var}(\tilde{g})$ of the stacked BLUPs. In this case, the *marginal* posterior trait covariance (obtained by averaging per-genotype trait blocks; Werner et al.) is used for index calculation. • A numeric matrix of dimension $n_{trait} \times n_{trait}$ giving the trait covariance matrix directly. If `var.mat` is `NULL`, the trait covariance matrix is estimated empirically as `cov(effects)`.
`gains`	Numeric vector of length n_{trait} specifying the desired gains for each trait. Used only for the desired gains index.
`weights`	Numeric vector of length n_{trait} specifying economic weights for each trait. Used only for the Smith–Hazel index.
`nthreads`	Integer. Number of OpenMP threads to use.

Details

- **Desired gains index** , based on user-specified desired trait gains and a trait (co)variance matrix.
- **Smith–Hazel index**, based on user-specified economic weights.

Exactly one of `gains` or `weights` must be provided.

Value

A list with components:

- `index`: A data.frame containing the selection index values for each genotype.
- `weights`: The trait weights used to construct the index.

`measure_cross_plan` *Evaluate a cross plan by averaging value metrics across all crosses (per trait)*

Description

Aligns `cross.df` to `crosses` (2-way or 4-way) and computes per-trait means across all crosses for available metrics (EGEBV, ETGV, SPV, TSPV).

Usage

```
measure_cross_plan(crosses, cross.df)
```

Arguments

`crosses`	data.frame with 2 columns (2-way) or 4 columns (4-way).
`cross.df`	data.frame (or coercible to data.frame) containing per-cross trait columns such as EGEBV#, ETGV#, SPV#, TSPV#. If parent columns are present (parent1..parent4 or male/female), they are used for alignment.

Value

A data.frame with one row per trait and columns: `trait`, `mean_EGEBV`, `mean_ETGV`, `mean_SPV`, `mean_TSPV`, `n_crosses`.

`measure_traits` *Summarize a multi-trait selection index*

Description

Computes summary statistics for a linear selection index given a trait covariance matrix and either (i) desired gains or (ii) fixed weights (Smith-Hazel).

Usage

```
measure_traits(var.mat = NULL, gains = NULL, weights = NULL, intensity = 1)
```

Arguments

`var.mat`	Numeric trait covariance matrix ($n_{trait} \times n_{trait}$).
`gains`	Optional numeric vector of desired gains (length n_{trait}). If supplied, index weights are computed as `solve(var.mat) %*% gains`.
`weights`	Optional numeric vector of index weights (length n_{trait}). Supply this to summarize an index with fixed weights.
`intensity`	Numeric scalar. Standardized selection differential i used to scale expected responses (default 1).

Details

For a trait covariance matrix G and index weights b, the index is $I = b^\top T$. The function returns the index variance $\sigma_I^2 = b^\top G b$, the expected standardized response in the index $i\sigma_I$ (where `i` is the standardized selection differential), the expected response in each trait $i\, Gb/\sigma_I$, and the correlation between each trait and the index.

If `gains` are provided, weights are computed as $b = G^{-1}d$, where d is the desired gains vector.

Value

A list with:

- `overall.df`: data.frame with `index.var` (σ_I^2) and `gain.index` ($i\sigma_I$)
- `traits.df`: data.frame with per-trait variance, correlation with the index, expected gain, and weight

`optimal_cross_selection` *Optimal cross selection via a genetic algorithm (with optional fixed crosses)*

Description

Uses a genetic algorithm (GA) to choose `ncrosses` parent pairs that balance an average criterion (utility, `criterion`) and genomic similarity (computed from `G.mat`) according to a target trade-off angle. If `fixed.crosses` is provided, those crosses are always included and the GA selects the remaining crosses from `crosses`.

Usage

```
optimal_cross_selection(
  crosses,
  fixed.crosses = NULL,
  ncrosses,
  target.angle,
  criterion,
  criterion.fixed = NULL,
  G.mat,
  return.params = FALSE,
  params = list(),
  nthreads = 4L
)
```

Arguments

`crosses`	Integer or character data.frame or matrix ($K \times 2$). Candidate *variable* crosses. Must have exactly two columns; selfings are not allowed.
`fixed.crosses`	Integer or character data.frame or matrix ($F \times 2$) or `NULL`. Crosses that must be included in the final plan. Can have 0 rows. Must use the same indexing/identifiers as `crosses`. Selfings are not allowed.
`ncrosses`	Integer. Total number of crosses in the final plan (variable + fixed). Must be $\geq$ `nrow(fixed.crosses)`.
`target.angle`	Numeric, target trade-off angle between utility and similarity. Smaller angles emphasize `criterion`; larger angles emphasize similarity control, allowed values are in the range of 0 to 90. Use 0 to prioritize criterion only, and 90 to prioritize diversity (G.mat). Everything in between is a trade-off.
`criterion`	Numeric vector (length = `nrow(crosses)`). Utility for each candidate cross in `crosses`.
`criterion.fixed`	Numeric vector (length = `nrow(fixed.crosses)`) or `NULL`. Utility for each fixed cross; if `fixed.crosses` has 0 rows, this can be length-0.
`G.mat`	Numeric square matrix ($n \times n$). Genomic relationship matrix used to evaluate similarity/diversity among parents. If `crosses` or `fixed.crosses` are character, `rownames(G.mat)` must be present.
`return.params`	Logical. If `TRUE`, return additional GA summary metrics.

`params` A list of GA parameters:

- `propability.mutate` Numeric in $[0, 1]$. Mutation probability.
- `n.mutate` Integer ≥ 0. Mutation magnitude parameter.
- `n.select` Integer ≥ 2. Number selected per generation.
- `n.pop` Integer ≥ 2. Population size.
- `max.generation` Integer ≥ 1. Maximum generations.
- `max.iteration` Integer ≥ 1. Max iterations without improvement.
- `angle.penalty` Numeric ≥ 0. Penalty for deviation from `target.angle`.

`nthreads` Integer ≥ 1. Number of threads to use.

Details

Both `crosses` and `fixed.crosses` may be provided either as integer indices (1-based, referring to rows/columns of `G.mat`) or as character identifiers matching `rownames(G.mat)`. I

The returned cross plan is assembled from the selected rows of `crosses` and all rows of `fixed.crosses`. If fixed crosses also appear among the candidate crosses, duplicates may occur in the final plan (a warning is issued).

Value

If `return.params = FALSE`**:** A data.frame with columns `parent1` and `parent2` describing the selected cross plan, returned in the same representation (integer indices or character IDs) as supplied in `crosses`/`fixed.crosses`.

If `return.params = TRUE`**:** A list with elements:

- `crossPlan` A data.frame with columns `parent1` and `parent2` describing the selected cross plan (mapped back to the original input representation).
- `uMax` Maximum attainable average utility when optimizing only utility.
- `uMin` Minimum attainable average utility under similarity optimization.
- `simMax` Similarity corresponding to `uMax`.
- `simMin` Similarity corresponding to `uMin`.
- `uBest` Average utility achieved by the optimized cross plan.
- `simBest` Similarity of the optimized cross plan.
- `angleBest` Angle (radians) of the optimized solution.
- `lenBest` Objective-space length of the optimized solution vector.

---

plot_cross_plan *Plot cross plan ridge distributions per trait*

Description

Creates ridgeline density plots of simulated cross outcomes, separated by trait. For each cross and trait, values are sampled from a Normal distribution whose mean is taken from the corresponding predicted value column and whose variance is taken from the available variance components.

Usage

```
plot_cross_plan(crosses, cross.df, traits = NULL, nsamples = 1000L)
```

Arguments

crosses	data.frame with 2 columns (2-way crosses) or 4 columns (4-way crosses), defining the parents in each cross. Rows are used to label and order crosses in the plot.
cross.df	data.frame (or object coercible to a data.frame) as created from the get_variance or get_optimal_haploid_value function containing per-cross trait columns such as EGEBV#, ETGV#, SPV#, TSPV#, OHV#, and variance components var#, var.A#, var.D#. If cross.df contains parent columns (e.g., parent1..parent4 or male,female), they are used to align rows to crosses.
traits	integer vector of trait indices to plot (e.g., c(1,3)), or NULL to plot all traits found in cross.df.
nsamples	integer. Number of Monte Carlo samples per cross and trait.

Details

Variance / mean handling (per trait):

- If var.A# is available, an *additive* ridge is drawn using mean = EGEBV# and sd = sqrt(var.A#).
- If var.A# and var.D# and ETGV# are available, an additional *additive+dominance* ridge is drawn using.
- If only var# is available (and no var.A#), a single ridge is drawn using mean = EGEBV# and sd = sqrt(var#).

Point overlays are added when the corresponding columns are present:

- EGEBV# (always used when available) and SPV# are shown as points and share the additive colour family.
- ETGV# and TSPV# are shown as points and share the additive+dominance colour family.
- OHV# is shown as a black cross (shape = 4).

A dotted vertical line indicates the per-trait average gain (mean of EGEBV# across crosses). The plot is faceted by trait (up to 3 columns).

Value

A ggplot object.

Index